

ui.page_scroller 全面详细解析

`ui.page_scroller` 是 NiceGUI 3.3.0+ 版本新增的滚动控制组件，基于 Quasar 框架的 PageScroller 组件封装，专注于实现“滚动到顶部”等快捷滚动功能。该组件默认以粘性定位（类似 `ui.page_sticky`）悬浮在页面指定位置，点击后可触发页面平滑滚动到顶部，支持滚动偏移、动画时长等自定义配置，是提升页面交互体验的实用组件。以下从核心特性、初始化参数、属性与方法、使用示例、注意事项等方面进行全面解析。

一、核心特性

- 快捷滚动功能：**默认绑定“滚动到顶部”逻辑，点击组件即可触发页面平滑滚动，无需手动编写 JavaScript 滚动代码。
- 粘性定位悬浮：**继承 `ui.page_sticky` 的基础定位能力，支持固定在页面角落（如底部右侧），滚动时保持可见。
- 滚动行为定制：**可配置滚动触发阈值、动画时长、滚动方向（正向 / 反向），适配不同交互需求。
- 样式高度灵活：**支持通过 `style`、`classes` 调整组件外观（如背景色、图标、圆角、阴影），与页面设计风格保持一致。
- 状态与事件支持：**继承基类的显示 / 隐藏、事件绑定等能力，可动态控制组件可见性，监听点击、滚动状态变化等事件。
- 版本兼容性：**仅支持 NiceGUI 3.3.0 及以上版本，依赖 Quasar 原生 PageScroller 组件能力。

二、初始化参数 (Initializer)

`ui.page_scroller` 的初始化参数包含定位配置（继承自 `ui.page_sticky`）和滚动行为配置，所有参数均为可选，默认值适配通用场景，详细说明如下：

参数名	类型	说明	默认值
定位相关参数（继承自 <code>ui.page_sticky</code> ）	-	-	-
<code>position</code>	str	粘性定位方位，可选值： <code>top-left</code> / <code>top-right</code> / <code>bottom-left</code> / <code>bottom-right</code> （指定组件悬浮位置）	未明确默认，常用 <code>bottom-right</code>
<code>x_offset</code>	int	水平偏移量（单位：像素），相对于定位方位的横向距离	0
<code>y_offset</code>	int	垂直偏移量（单位：像素），相对于定位方位的纵向距离	0
<code>z_index</code>	int	组件层级优先级（ <code>z-index</code> ），数值越大越靠上层，避免被其他元素遮挡	1000
滚动行为相关参数	-	-	-
<code>scroll_offset</code>	int	滚动触发阈值（单位：像素）：页面滚动距离超过该值时，组件才显示（未超过时隐藏）	未明确默认，常用 200

参数名	类型	说明	默认值
duration	int/float	滚动动画时长（单位：毫秒），数值越大滚动越平缓	未明确默认，常用 500
reverse	bool	滚动方向：- <code>False</code> （默认）：滚动到顶部- <code>True</code> ：滚动到底部（反向滚动）	False

三、核心属性 (Properties)

继承自 `Element`、`Visibility` 及 `ui.page_sticky` 相关基类，支持动态获取和修改组件状态与配置，核心属性如下：

属性名	类型	说明
classes	Classes[Self]	组件的 HTML 类名，支持通过 Tailwind/Quasar 类调整样式（如 <code>rounded-full shadow-lg</code> ），支持链式调用修改
client	Client	组件所属的客户端实例，关联 websocket 连接、布局上下文等
html_id	str	组件在 HTML DOM 中的唯一 ID（2.16.0+ 版本支持），可用于直接操作 DOM
is_deleted	bool	组件是否已被删除（只读属性），用于判断生命周期状态
is_ignoring_events	bool	组件是否正在忽略事件（只读属性），用于事件控制逻辑
props	Props[Self]	组件的 Quasar 原生属性，支持链式调用添加 / 删除（如 <code>props('flat')</code> ），扩展组件功能
style	Style[Self]	组件的内联 CSS 样式，支持链式调用（如 <code>style('background-color: #42b983').style('color: white')</code> ），精准控制外观
visible	BindableProperty	组件的可见性（ <code>True</code> 显示 / <code>False</code> 隐藏），可绑定数据动态控制
scroll_offset	int	滚动触发阈值，可动态修改（如 <code>scroller.scroll_offset = 300</code> ）
duration	int/float	滚动动画时长，可动态修改（如 <code>scroller.duration = 800</code> ）
reverse	bool	滚动方向，可动态切换（如 <code>scroller.reverse = True</code> 改为滚动到底部）

四、关键方法 (Methods)

`ui.page_scroller` 继承了基类的丰富方法，同时包含滚动相关专属方法，按功能分类解析核心方法：

1. 基础状态操作

方法名	参数	说明
show()	-	显示组件
hide()	-	隐藏组件
toggle()	-	切换组件的显示 / 隐藏状态
delete()	-	删除组件及所有子元素，释放资源
update()	-	同步组件状态到客户端，修改样式、属性后需调用以生效

2. 样式与属性修改

方法名	核心参数	说明
style()	css_str	设置内联样式，支持多轮链式调用（如 <code>style('width: 40px').style('height: 40px')</code> ）
classes()	class_str	设置 HTML 类名，结合 Tailwind 实现快速样式（如 <code>classes('bg-blue-500 text-white p-2 rounded-full')</code> ）
props()	props_str	设置 Quasar 原生属性（如 <code>props('shadow-md')</code> 添加中等阴影、 <code>props('icon=arrow_up')</code> 指定图标）
set_scroll_offset()	offset: int	动态修改滚动触发阈值（如 <code>set_scroll_offset(300)</code> ，页面滚动超过 300px 才显示组件）
set_duration()	duration: int/float	动态修改滚动动画时长（如 <code>set_duration(1000)</code> ，滚动动画持续 1 秒）
set_reverse()	reverse: bool	动态切换滚动方向（如 <code>set_reverse(True)</code> ，从“滚动到顶部”改为“滚动到底部”）

3. 数据绑定方法

支持将组件状态（如 `visible`）与 Python 对象属性动态绑定，实现单向 / 双向同步：

方法名	核心参数	说明
bind_visibility()	target_object, target_name	双向绑定：组件可见性与目标对象属性同步（一方修改，另一方自动更新）
bind_visibility_from()	target_object, target_name	单向绑定（从目标到组件）：目标对象属性变化时，同步更新组件可见性
bind_visibility_to()	target_object, target_name	单向绑定（从组件到目标）：组件可见性变化时，同步更新目标对象属性

4. 事件绑定方法

方法名	核心参数	说明
on()	type, handler, js_handler	绑定组件事件（如 <code>click</code> 点击事件、 <code>scroll</code> 滚动事件），支持 Python 处理器或 JavaScript 处理器（2.18.0+ 版本可同时指定）
on_click()	callback	快捷绑定点击事件（如自定义点击后的滚动逻辑，覆盖默认行为）
on_scroll()	callback	绑定滚动状态变化事件（如滚动开始、滚动结束时触发回调）

5. 滚动专属方法

方法名	核心参数	说明
trigger_scroll()	-	手动触发滚动（无需点击组件，直接执行滚动逻辑）

五、使用示例

1. 基础用法：默认滚动到顶部按钮

```
from nicegui import ui

@ui.page('/basic_scroller')
def basic_scroller_demo():
    # 创建底部右侧悬浮的滚动到顶部按钮
    with ui.page_scroller(
        position='bottom-right', # 定位在底部右侧
        x_offset=20, # 水平偏移 20px
        y_offset=20, # 垂直偏移 20px
        scroll_offset=200, # 页面滚动超过 200px 才显示
        duration=500 # 滚动动画时长 500ms
    ).classes('rounded-full shadow-lg'):
        # 自定义按钮样式（绿色背景、白色图标）
        ui.button(icon='arrow_upward').props('flat color=white bg-green-500 hover:bg-green-600')

    # 页面主体内容（超长文本用于测试滚动）
    with ui.column().classes('p-6'):
        ui.label('滚动到顶部按钮演示').classes('text-2xl font-bold mb-4')
        [ui.label(f'内容行 {i}：滚动超过 200px 后显示悬浮按钮') for i in range(200)]

ui.run()
```

2. 高级用法：动态切换滚动方向（顶部 / 底部）

```
from nicegui import ui

@ui.page('/dynamic_scroller')
def dynamic_scroller_demo():
```

```

# 创建滚动组件（默认滚动到顶部）
scroller = ui.page_scroller(
    position='bottom-left',
    x_offset=20,
    y_offset=20,
    scroll_offset=150,
    duration=800
).classes('rounded-full shadow-md')

with scroller:
    ui.button(icon='arrow_upward', id='scroll-btn').props('flat color=white bg-blue-500')

# 切换滚动方向的逻辑
is_scroll_to_top = True
def toggle_scroll_direction():
    nonlocal is_scroll_to_top
    is_scroll_to_top = not is_scroll_to_top
    scroller.set_reverse(not is_scroll_to_top) # reverse=False 滚动到顶部, True 滚动到底部
    # 更新按钮图标
    scroll_btn = ui.query('#scroll-btn').first()
    scroll_btn.props('icon=arrow_downward' if not is_scroll_to_top else 'arrow_upward')
    ui.notify(f'已切换为{"滚动到底部" if not is_scroll_to_top else "滚动到顶部"}')

# 页面主体内容
with ui.column().classes('p-6'):
    ui.label('动态切换滚动方向演示').classes('text-2xl font-bold mb-4')
    ui.button('切换滚动方向', on_click=toggle_scroll_direction).props('elevated')
    [ui.label(f'内容行 {i}') for i in range(150)]

ui.run()

```

3. 实用场景：自定义样式与滚动阈值

```

from nicegui import ui

@ui.page('/custom_scroller')
def custom_scroller_demo():
    # 创建自定义样式的滚动组件
    with ui.page_scroller(
        position='top-right',
        x_offset=15,
        y_offset=15,
        scroll_offset=300, # 滚动超过 300px 显示
        duration=600,
        z_index=1500 # 提高层级, 避免被导航栏遮挡
    ).style('background-color: #f59e0b; border-radius: 8px; padding: 8px'):
        ui.icon('arrow_up', color='white', size='24px') # 直接使用图标组件, 无需按钮

# 页面主体内容
with ui.column().classes('p-6'):
    ui.label('自定义样式滚动组件演示').classes('text-2xl font-bold mb-4')

```

```
[ui.label(f'内容行 {i}: 滚动超过 300px 显示橙色悬浮图标') for i in range(250)]
```

```
ui.run()
```

4. 数据绑定：联动组件可见性

```
from nicegui import ui

class AppState:
    def __init__(self):
        self.show_scroller = True # 控制滚动组件可见性

@ui.page('/bind_scroller')
def bind_scroller_demo():
    state = AppState()

    # 创建滚动组件并绑定可见性
    scroller = ui.page_scroller(
        position='bottom-right',
        x_offset=20,
        y_offset=20,
        scroll_offset=200
    ).bind_visibility(state, 'show_scroller').classes('rounded-full shadow-lg')

    with scroller:
        ui.button(icon='arrow_upward').props('flat color=white bg-purple-500')

    # 页面主体控制区
    with ui.column().classes('p-6'):
        ui.label('滚动组件可见性绑定演示').classes('text-2xl font-bold mb-4')
        ui.checkbox('显示滚动组件', value=state.show_scroller).bind_value(state,
'show_scroller')
        ui.button('隐藏', on_click=lambda: scroller.hide()).props('flat')
        ui.button('显示', on_click=lambda: scroller.show()).props('flat')
        [ui.label(f'内容行 {i}') for i in range(150)]

ui.run()
```

六、注意事项

- 版本依赖：**必须使用 NiceGUI 3.3.0 及以上版本，低版本会报错，需通过 `pip install --upgrade nicegui` 升级。
- 定位与遮挡：**
 - 建议将 `z_index` 设置为高于页面其他固定元素（如 `ui.header`、`ui.footer`），避免被遮挡。
 - 定位方位优先选择 `bottom-right` 或 `bottom-left`，符合用户操作习惯（底部悬浮按钮不易误触）。
- 滚动阈值适配：**`scroll_offset` 建议设置为 200-300px，过小会导致组件频繁显示 / 隐藏，过大则用户需滚动较远才能看到组件。
- 样式定制技巧：**

- 若需圆形悬浮按钮，可使用 `classes('rounded-full')` 配合固定宽高（如 `style('width: 48px; height: 48px')`）。
 - 图标组件（`ui.icon`）比按钮更简洁，适合极简风格设计，可直接作为 `ui.page_scroller` 的子元素。
5. **反向滚动限制**： `reverse=True` 时滚动到底部，需确保页面有足够的滚动高度（内容超过视口），否则滚动逻辑不生效。
6. **事件覆盖**：若通过 `on_click()` 绑定自定义点击事件，会覆盖默认滚动逻辑，需手动调用 `trigger_scroll()` 或编写自定义滚动代码。

七、扩展参考

- **Quasar 原生文档**： `ui.page_scroller` 基于 Quasar PageScroller 组件，更多原生属性可通过 `props()` 传递，详细参考 [Quasar PageScroller 文档](#)。
- **滚动动画优化**： `duration` 参数建议设置为 300-1000ms，过短会导致滚动生硬，过长会影响交互效率。
- **响应式适配**：小屏幕下可通过 `classes('md:block sm:hidden')` 隐藏组件，或调整 `x_offset / y_offset` 减小偏移量，避免布局拥挤。

NiceGUI 中 `ui.page_scroller` 的全维度解析

`ui.page_scroller` 是 NiceGUI 专为**页面滚动控制**设计的组件，核心作用是在页面中生成一个“滚动触发按钮 / 区域”，点击后可快速滚动到页面指定位置（如顶部、底部、自定义锚点），是提升用户体验的高频组件，常与 `ui.page_sticky` 配合实现“返回顶部”“跳转到指定区块”等功能。

一、核心作用与适用场景

1. 核心作用

- 一键触发页面滚动到预设位置（顶部 / 底部 / 自定义锚点）；
- 支持自定义滚动动画时长、触发元素样式、显示 / 隐藏逻辑；
- 可绑定到任意 UI 元素（如按钮、图标），也可使用组件内置的默认样式。

2. 典型适用场景

场景	示例
返回顶部按钮	页面滚动后，右下角显示“回到顶部”按钮，点击返回页面顶端
跳转到指定区块	导航栏点击“功能介绍”，滚动到页面中对应的内容区块
滚动到底部	长列表 / 日志页面中，一键滚动到最新内容位置
锚点导航	多区块页面中，快速切换到不同内容区域

二、基本语法与使用方式

1. 基础语法

```
from nicegui import ui

# 基础用法：滚动到顶部（默认）
ui.page_scroller()

# 完整参数示例
ui.page_scroller(
    target: str | None = 'top', # 滚动目标（top/bottom/锚点ID）
    duration: float = 0.5,      # 滚动动画时长（秒）
    offset: int = 0,            # 滚动偏移量（像素，如离顶部20px）
    visible: bool = True,       # 是否可见
    # 自定义触发元素（替代默认样式）
    icon: str = 'arrow_upward', # 内置图标（Material Icons）
    color: str = 'primary',     # 颜色主题（primary/secondary/info等）
)

# 绑定到自定义元素（推荐）
with ui.page_sticky(position='bottom-right', offset=20):
    ui.page_scroller(target='top').props('fab round') # 悬浮圆形按钮样式
```

2. 核心参数详解

参数名	类型	取值说明	默认值
target	str / None	滚动目标：- 'top'：页面顶部- 'bottom'：页面底部- 字符串（如 '#section1'）：指定 ID 的锚点元素- None：需手动绑定滚动目标	'top'
duration	float	滚动动画时长（秒），0 表示无动画（瞬间滚动）	0.5
offset	int	滚动到目标位置后的偏移量（像素）：- 正数：向下偏移（如 target='top' 时，offset=20 表示离顶部 20px）- 负数：向上偏移	0
visible	bool	初始是否可见，可通过 .set_visible() 动态修改	True
icon	str	内置触发按钮的图标（Material Icons 名称，如 arrow_upward / arrow_downward）	arrow_upward
color	str	按钮颜色主题（遵循 NiceGUI 内置主题：primary/secondary/info/success/warning/error）	primary

3. 基础示例（返回顶部按钮）

```
from nicegui import ui

# 结合ui.page_sticky实现悬浮返回顶部按钮
with ui.page_sticky(position='bottom-right', offset=20):
    # 滚动到顶部，动画时长0.3秒，悬浮圆形按钮样式
    ui.page_scroller(target='top', duration=0.3).props('fab round')
```



```
# 页面主体内容（用于测试滚动）
ui.label('页面顶部').classes('text-3xl text-center my-5')
for i in range(30):
    ui.card(f'内容卡片 {i+1}').classes('w-full my-2')
ui.label('页面底部').classes('text-3xl text-center my-5')

ui.run()
```

4. 进阶示例（锚点导航）

```
from nicegui import ui

# 顶部导航栏（点击跳转到对应锚点）
with ui.page_sticky(position='top', offset=0).classes('bg-white p-2 shadow-md w-full'):
    with ui.row():
        # 滚动到section1锚点，偏移量-20（避免被导航栏遮挡）
        ui.page_scroller(target='#section1', offset=-20).props('flat').text('功能介绍')
        ui.page_scroller(target='#section2', offset=-20).props('flat').text('使用教程')
        ui.page_scroller(target='#section3', offset=-20).props('flat').text('常见问题')

# 页面内容区块（设置锚点ID）
ui.section().props('id=section1').classes('py-10')
ui.label('功能介绍').classes('text-2xl mb-5')
ui.div().classes('h-60 bg-gray-100 mb-5') # 占位内容

ui.section().props('id=section2').classes('py-10')
ui.label('使用教程').classes('text-2xl mb-5')
ui.div().classes('h-60 bg-gray-100 mb-5')

ui.section().props('id=section3').classes('py-10')
ui.label('常见问题').classes('text-2xl mb-5')
ui.div().classes('h-60 bg-gray-100 mb-5')

# 右下角返回顶部按钮
with ui.page_sticky(position='bottom-right', offset=20):
    ui.page_scroller(target='top', icon='arrow_upward').props('fab round color=blue')

ui.run()
```

三、关键特性与注意事项

1. 滚动动画与性能

- `duration` 越大，滚动动画越平缓，但过长可能影响体验（建议 0.3-1 秒）；
- 设为 0 时无动画，适合需要快速定位的场景（如日志页面跳到底部）；
- 滚动动画基于 CSS `scroll-behavior` 实现，兼容所有现代浏览器。

2. 锚点定位的注意事项

- 锚点元素需通过 `.props('id=xxx')` 设置唯一 ID (如 `ui.section().props('id=section1')`) ;
- 目标锚点 ID 需以 `#` 开头 (如 `target='#section1'`) , 否则会被识别为普通字符串;
- 若锚点元素被隐藏 (`visible=False`) 或不在可视区域, 滚动可能失效;
- `offset` 参数可解决“锚点被固定导航栏遮挡”的问题 (通常设为负的导航栏高度) 。

3. 自定义触发元素

`ui.page_scroller` 默认生成带图标按钮, 可通过以下方式自定义样式 / 内容:

```
from nicegui import ui

with ui.page_sticky(position='bottom-right', offset=20):
    # 自定义触发元素: 带文字的按钮
    scroller = ui.page_scroller(target='top', duration=0.4)
    scroller.props('color=green').text('回到顶部') # 替换默认图标为文字

# 或嵌套自定义内容
with ui.page_sticky(position='bottom-right', offset=20):
    with ui.page_scroller(target='bottom') as scroller:
        with ui.row().classes('bg-white p-2 rounded-lg shadow'):
            ui.icon('arrow_downward').classes('mr-1')
            ui.label('到底部')
```

4. 动态控制滚动目标

可通过组件引用动态修改滚动目标、可见性等属性:

```
from nicegui import ui

# 保存组件引用
scroller = ui.page_scroller(target='top', visible=False)

# 滚动到一定位置后显示返回顶部按钮
def on_scroll(e):
    # e.scroll_top 是当前滚动距离顶部的像素值
    scroller.set_visible(e.scroll_top > 300)

# 绑定页面滚动事件
ui.page().on('scroll', on_scroll)

# 主体内容
for i in range(40):
    ui.div().classes('h-20 bg-gray-50 my-1')

ui.run()
```

5. 与 ui.scroll_to 的区别

ui.page_scroller 是**组件化**的滚动控制（自带触发元素），而 ui.scroll_to 是**函数式**的滚动 API（需手动绑定触发事件），二者可互补：

特性	ui.page_scroller	ui.scroll_to
形式	可视化组件	底层函数
触发方式	点击组件自动触发	手动调用函数
自定义程度	中等（可改样式）	高（完全自定义）
适用场景	固定触发按钮	动态逻辑滚动（如异步任务完成后滚动）

示例：结合 ui.scroll_to 实现异步任务完成后滚动到结果区域

```
from nicegui import ui
import asyncio

# 结果区域锚点
result_section = ui.section().props('id=result').classes('mt-10')
result_label = ui.label('结果将显示在这里').classes('text-xl')

async def async_task():
    await asyncio.sleep(2)
    result_label.set_text('异步任务完成！这是结果内容')
    # 手动调用scroll_to滚动到结果区域
    await ui.scroll_to(target='#result', duration=0.5, offset=-20)

ui.button('执行任务并滚动到结果', on_click=async_task)

# 主体内容
for i in range(20):
    ui.div().classes('h-20 bg-gray-50 my-1')

ui.run()
```

四、实战场景示例

1. 智能返回顶部按钮（滚动显示 / 隐藏）

```
from nicegui import ui

# 初始化返回顶部按钮（默认隐藏）
with ui.page_sticky(position='bottom-right', offset=20):
    scroller = ui.page_scroller(target='top', duration=0.3, visible=False)
    scroller.props('fab round color=primary')

# 监听页面滚动事件，动态显示/隐藏按钮
```

```
def handle_scroll(event):
    # 滚动距离超过300px时显示
    scroller.set_visible(event.scroll_top > 300)

ui.page().on('scroll', handle_scroll)

# 页面内容
ui.label('滚动页面查看返回顶部按钮').classes('text-2xl text-center my-5')
for i in range(40):
    ui.card(f'内容项 {i+1}').classes('w-full my-2')

ui.run()
```

2. 双向滚动按钮（顶部 / 底部切换）

```
from nicegui import ui

# 跟踪当前滚动状态
is_at_top = True

def toggle_scroll():
    global is_at_top
    if is_at_top:
        scroller.target = 'bottom'
        scroller.icon = 'arrow_downward'
    else:
        scroller.target = 'top'
        scroller.icon = 'arrow_upward'
    is_at_top = not is_at_top
    # 手动触发滚动（因修改target后需点击才生效，这里直接调用scroll_to）
    ui.scroll_to(target=scroller.target, duration=0.5)

with ui.page_sticky(position='bottom-right', offset=20):
    scroller = ui.page_scroller(target='top', duration=0.4)
    scroller.props('fab round').on('click', toggle_scroll)

# 页面内容
for i in range(50):
    ui.div().classes('h-15 bg-gray-100 my-1')

ui.run()
```